FACEATTEND · STUDY NOTES

Transfer Learning & the AI Models

What transfer learning is (in simple words), where it is used in this project, which 5 AI models are used, where each one works, and what each one does at that place.

1. Transfer learning in simple words
2. The two types — and which one we use
3. Is transfer learning used in my project? (Yes)
4. The 5 models at a glance
5. Model 1 — SSD MobileNet v1 (face detector)
6. Model 2 — Face Landmark 68 (face points)
7. Model 3 — Face Recognition ResNet (face fingerprint)
8. Models 4 & 5 — MiniFASNetV2 + V1SE (anti-spoofing)
9. Where the models work: kiosk flow
10. Where the models work: registration flow
11. Registration is NOT training
12. Why transfer learning here (benefits & limits)
13. Viva / exam questions & answers
14. Remember it in 60 seconds

1. Transfer learning in simple words

Transfer learning = reuse a model that already learned one big job, for a new related job, instead of training from zero.

Easy example: learning to drive

Someone who can already **ride a bicycle** knows balance, traffic rules and steering.

→

They learn a **motorbike** much faster. They **transfer** what they already know.

Same idea for faces

A model was trained on **millions of faces of other people**. It learned what makes faces different: eye shape, nose, jaw, spacing.

→

We reuse that model to recognize **our own users**, people it has **never seen**, without training it again.

Without vs. with transfer learning

	Train from zero	Transfer learning (our project)
Data needed	Millions of labelled face photos	**5 photos per person**
Time	Days or weeks on GPUs	**Seconds** (just register)
Hardware	Powerful GPU	**Normal laptop CPU**
Add a new person	Retrain the model	**Just save their template**

Remember: "**Learned once, reused everywhere.**" The heavy learning was done by the model's authors. We only **use** the knowledge.

2. The two types — and which one we use

Type 1 — Feature extraction (we use this)

- Use the pre-trained model **exactly as it is** (frozen).
- Take its output (the "features") and use it for your task.
- **No training** at all.

Our project: the ResNet gives 128 numbers per face. We compare those numbers to find who it is.

Type 2 — Fine-tuning (NOT used)

- Take the pre-trained model and **train it a little more** on your own data.
- Changes the model's weights.
- Needs a dataset, time and usually a GPU.

Our project never changes any model weights.

Picture of feature extraction

Face photo → Pre-trained model (frozen, never changes) → 128 numbers ("face fingerprint") → Compare distance → Who is it?

Memory trick: "Freeze & Use". We **freeze** the model (no training) and **use** its output. That is feature extraction.

Related words you may hear

Word	Meaning in simple words	In our project
Pre-trained model	A model someone already trained and shared	All 5 models
Embedding / descriptor	A list of numbers that describes something	128 numbers per face
Metric learning	Training so that "same = close, different = far" in numbers	How the ResNet was trained by its authors
Few-shot recognition	Recognizing someone from only a few examples	5 photos per person are enough
Inference	Using a model to get an answer (no learning)	Everything the backend does

3. Is transfer learning used in my project?

Yes. The project uses **transfer learning by feature extraction**. It uses **5 pre-trained models** without any fine-tuning. **No model is trained** anywhere in the project.

Question	Answer
Is any model trained in the project?	**No.** All models come pre-trained.
Is fine-tuning done?	**No.** Model weights never change.
Where is the "transfer"?	Knowledge learned from other people's faces is reused to recognize **our** users, and to detect faces, face points and fake faces.
Where are the models?	Face models: inside the npm package `@vladmandic/face-api` (folder `node_modules/@vladmandic/face-api/model`). Anti-spoof models: `backend/models/antispoof/`.
Which code loads them?	`backend/src/services/faceEngine.js` (face models) and `backend/src/services/liveness.js` (anti-spoof models). Both load once when the server starts.
What runs them?	TensorFlow.js (WASM backend) for face models; onnxruntime-node for anti-spoof models.

What our project adds on top (our own logic, not learned)

- **Quality rules**: face size, blur, lighting, pose (`quality.js`)
- **Matching rule**: distance ≤ 0.5 and 0.06 better than the 2nd-best person (`matcher.js`)
- **Decision steps**: 3 stable frames, head-turn challenge, confirm (`scanSession.js`)
- **Attendance rules**: one record per person per day (`attendanceService.js`)

Easy line: "The models give the **knowledge**; our code gives the **rules**."

4. The 5 models at a glance

#	Model	Nickname	Job	Input → Output	Size	Runs with
1	SSD MobileNet v1	**The Finder**	Find faces	Camera frame → face boxes + score	5.6 MB	TensorFlow.js
2	Face Landmark 68	**The Pointer**	Mark 68 face points	Face crop → 68 (x, y) points	0.36 MB	TensorFlow.js
3	Face Recognition (ResNet)	**The Fingerprinter**	Describe the face	Aligned face → 128 numbers	6.4 MB	TensorFlow.js
4	MiniFASNetV2	**Guard 1**	Real or fake? (close crop)	80×80 crop (2.7× face) → real/fake score	1.7 MB	onnxruntime
5	MiniFASNetV1SE	**Guard 2**	Real or fake? (wide crop)	80×80 crop (4.0× face) → real/fake score	1.7 MB	onnxruntime

Memory trick: "Find → Point → Fingerprint → Guard".
Finder finds the face · Pointer marks the eyes/nose · Fingerprinter makes 128 numbers · Guards check it's a real person.

Order they run on one camera frame

Camera frame → 1 Finder → 2 Pointer → quality check (our rules) → 4 + 5 Guards → 3 Fingerprinter → match (our rule)

Expensive models (Guards, Fingerprinter) only run if the frame passes the quality check, so bad frames are rejected quickly.

Who made them (the "learned once" part)

Model	Trained by	Learned from
1, 2, 3	face-api.js authors (maintained today in @vladmandic/face-api)	Large public face datasets (face boxes, face points, many thousands of different people)
4, 5	Minivision (Silent-Face-Anti-Spoofing project, Apache-2.0 licence)	Photos of real faces and of attacks (printed photos, screens)

5. Model 1 — SSD MobileNet v1 ("The Finder")

1 Face detector

Job	Find every face in the camera frame and draw a box around it.
Input	The whole camera frame (e.g. 640×480 pixels).
Output	List of face boxes (x, y, width, height) + confidence score (0–1).
Type	SSD = "Single Shot Detector" (finds all faces in one pass). MobileNet = light, fast network made for phones/CPUs.
File	`ssd_mobilenetv1_model.bin` (5.6 MB) in @vladmandic/face-api
Loaded in	`faceEngine.js → init()` at server start
Called in	`faceEngine.js → FrameImage.detect()`, used by `faceAnalysis.js`
Setting	`DETECTOR_MIN_CONFIDENCE = 0.5` (ignore weak detections)

What it does at that place

Place	Work done
Kiosk scan (every frame)	Finds faces. 0 faces → "Looking for face...". 2+ faces → "Please ensure only one person is visible." The box is also drawn on the kiosk screen.
Registration (every frame)	Same, but **any** second face blocks registration ("Only one person should be visible during registration").
Quality gate	Box size → "Move closer" / "Move back". Box at edge → "Please center your face". Low score → "Make sure your whole face is clearly visible".
Tracking	Box position is compared frame to frame. If the face jumps, progress resets (so two people can't be mixed).

Transfer here: it learned "what a face looks like" from thousands of labelled photos. We reuse that to find faces in our camera. No training by us.

6. Model 2 — Face Landmark 68 ("The Pointer")

2 68-point face landmarks

Job	Put 68 points on the face: jaw (0–16), eyebrows (17–26), nose (27–35), eyes (36–47), mouth (48–67).
Input	The face area found by Model 1.
Output	68 (x, y) coordinates.
File	`face_landmark_68_model.bin` (0.36 MB)
Loaded in	`faceEngine.js → init()`
Called in	`faceEngine.js → detect()` (runs right after Model 1)
Maths on the points	`faceGeometry.js → headPose()`

What it does at that place

Place	Work done with the 68 points
Alignment (before Model 3)	Finds the exact face area, so the Fingerprinter always sees the face in the same position. Better alignment = better recognition.
Head turn (yaw)	Nose tip vs. jaw edges. Too turned → "Please look directly at the camera".
Head tilt (roll)	Angle of the line between the eyes. > 20° → "Please keep your head straight".
Up/down (pitch)	Nose height between eyes and chin → "Please look straight at the camera, not up or down".
Liveness challenge	Checks you **really turned** left/right when asked, then looked back. A photo cannot do this.
Registration poses	Checks "turn slightly left" / "right" photos are really turned (0.15–0.65).
Eye openness (EAR)	Used for the optional "blink" challenge.

Transfer here: it learned where eyes/nose/jaw are from labelled face images. Our code turns those points into pose checks and the liveness challenge.

7. Model 3 — Face Recognition ResNet ("The Fingerprinter")

3 128-D face descriptor (the main transfer-learning model)

Job	Turn a face into **128 numbers** (a "face fingerprint" / embedding).
Input	The aligned face (using Model 2's points), resized to 150×150.
Output	128 decimal numbers.
Type	ResNet (deep network with "shortcut" connections), trained with **metric learning**: the same person's faces → close numbers, different people → far apart.
File	`face_recognition_model.bin` (6.4 MB)
Loaded in	`faceEngine.js → init()`
Called in	`faceEngine.js → FrameImage.descriptor()`
Numbers used by	`matcher.js` (compare), `gallery.js` (all users in memory), `biometricCrypto.js` (encrypt before saving)

How recognition works with it

New face → ResNet → 128 numbers → distance to each registered person → smallest ≤ 0.5? → 0.06 better than 2nd? → Same on 3 frames? → Identified

Place	Work done
Registration	Makes 128 numbers for each of the 5 photos → checks they are the same person → checks no one else already has this face (duplicate check) → encrypted and saved.
Kiosk	Makes 128 numbers for each good frame → compared with all saved templates → name or "Face not recognized".
Tracking	Numbers must stay similar inside one session, so a different person stepping in resets progress.
Duplicates tab	Compares everyone with everyone to find the same person registered twice.

Why this is the heart of transfer learning: it was trained on thousands of **other** people, yet it works for **your** users, because it learned **how to describe any face**, not just the people in its training set.

8. Models 4 & 5 — MiniFASNetV2 + MiniFASNetV1SE ("The Guards")

4 + 5 Passive anti-spoofing (liveness)

Job	Decide if the face is a **real live person** or a **fake** (printed photo, phone or monitor screen).
Input	80×80 crop around the face. Guard 1 (V2) uses a crop 2.7× the face; Guard 2 (V1SE) uses a wider 4.0× crop, so it also sees the edges of a phone or paper.
Output	3 scores → softmax → "real" probability (0–1). We average both guards.
Type	MiniFAS = "Mini Face Anti-Spoofing" network, very small and fast (~5 ms). SE = "Squeeze-and-Excitation" attention blocks.
Files	`backend/models/antispoof/MiniFASNetV2.onnx` , `MiniFASNetV1SE.onnx` (1.7 MB each)
Loaded in	`liveness.js → init()` using onnxruntime-node
Called in	`liveness.js → passiveScore()` ; crop made by `faceEngine.js → antispoofInput()`
Settings	`LIVENESS_THRESHOLD = 0.6` (average needed), `LIVENESS_HARD_FAIL = 0.2`

What they do at that place

Place	Work done
Kiosk	Score every good frame. Average of 3+ frames must be ≥ 0.6. Two frames below 0.2 → "Unable to confirm a live person." and a security event in the audit log.
Registration	Each photo must be live, so nobody can register someone using their photo.

Two layers of liveness

Passive (the models)
Looks at texture, glare, edges and moiré patterns of screens/paper. No action needed from the user.

Active (our code + Model 2)
"Turn your head slightly left/right". The landmark points prove the head really moved.

Transfer here: the Guards learned "real vs. fake" from Minivision's dataset of real and attack photos. We reuse them unchanged.

Measured on your own webcam: real faces scored 0.89–0.998. Attack scores must still be measured with `npm run evaluate-liveness` .

9. Where the models work: kiosk flow

Page `/kiosk` sends a frame every ~0.3 s to `POST /api/kiosk/scan` → `routes/kiosk.js` → `faceAnalysis.js` → `scanSession.js`.

Step	Screen shows	Model used	Code file
1. Find face	Looking for face...	**1 Finder**	faceEngine.detect
2. Face points	(face box appears)	**2 Pointer**	faceEngine.detect, faceGeometry
3. Quality	Move closer / Improve lighting ...	1 + 2 outputs + our rules	quality.js
4. Real or fake	Checking liveness...	**4 + 5 Guards**	liveness.passiveScore
5. Who is it	Verifying identity...	**3 Fingerprinter**	faceEngine.descriptor → matcher
6. Challenge	Please turn your head slightly to your left	**2 Pointer**	liveness.Challenge
7. Confirm	Verifying identity...	**3 Fingerprinter**	scanSession (CONFIRM)
8. Save	Attendance marked successfully.	no model	attendanceService

Memory trick for the kiosk: "F-P-Q-G-F-C": **F**inder → **P**ointer → **Q**uality → **G**uards → **F**ingerprinter → **C**hallenge.

10. Where the models work: registration flow

Page `/admin/students/new` → `FaceCapture.js` → `POST /api/students/check-sample` (live guide) and `POST /api/students` (save) → `enrollmentService.js`.

Step	Model used	What happens
Live guide (every frame)	1 Finder, 2 Pointer, 4 + 5 Guards	Exactly one face? Good quality? Correct pose (straight/left/right)? Live? Auto-capture after 2 good frames.
Save: re-check each photo	1, 2, 4, 5	Server checks all 5 photos again (never trusts the browser).
Save: make templates	**3 Fingerprinter**	128 numbers for each photo.
Same-person check	3 (numbers)	All 5 photos must be close to each other (≤ 0.55).
Duplicate check	3 (numbers)	Closer than 0.42 to someone already registered → warning.
Store	no model	Numbers encrypted (AES-256) and saved. Photos deleted from memory.

Which model is used where (summary)

Model	Kiosk	Registration	Duplicates tab	Evaluation scripts
1 Finder	✓	✓	—	✓
2 Pointer	✓ (quality + challenge)	✓ (quality + pose)	—	✓
3 Fingerprinter	✓ (identify)	✓ (templates)	✓ (uses saved numbers)	✓ (FAR/FRR)
4 + 5 Guards	✓	✓	—	✓ (APCER/BPCER)

11. Registration is NOT training

Common mistake: "When I register a person, the model learns their face." **Wrong.** The model does not change at all.

What really happens at registration

1. The frozen Fingerprinter reads the 5 photos.
2. It outputs 5 × 128 numbers.
3. The numbers are encrypted and saved in MongoDB (`students.samples`).
4. They are loaded into memory (`gallery.js`) for fast matching.

What really happens at the kiosk

1. The same frozen model makes 128 numbers for the new face.
2. `matcher.js` measures the distance to every saved person.
3. Small enough and clearly the best → that person.

	Training (not in our project)	Registration (our project)
Model weights	Change	**Never change**
Time	Hours/days	**~2 seconds**
Stored result	A new model file	**128 numbers per photo**
Delete a person	Retrain	**Delete their numbers**

Easy line: "Registration **stores** a face fingerprint; it does not **teach** the model."

12. Why transfer learning here (benefits & limits)

Benefits

- **No big dataset** needed: 5 photos per person.
- **No GPU**: runs on a normal laptop CPU (~0.3–0.4 s per frame).
- **Instant new users**: no retraining.
- **Proven accuracy**: the models were trained and tested by their authors on large datasets.
- **Privacy**: we store only 128 numbers, not photos.
- **Small**: all 5 models together ≈ 16 MB.

Limits (be honest)

- The models were trained on other data; accuracy on **your camera/lighting** must be measured (`npm run evaluate`).
- Very similar faces (twins) can look alike → our margin rule answers "Unable to verify identity" instead of guessing.
- One normal webcam can't stop every attack (e.g. 3D masks).
- Fine-tuning could improve accuracy, but needs a labelled dataset and a GPU.

Numbers to remember

5 pre-trained models	**128** numbers per face	**0.5** max match distance	**0.6** min liveness score
68 face points	**80×80** anti-spoof input	**0.06** margin over 2nd best	**0** models trained by us

13. Viva / exam questions & answers

Q1. What is transfer learning?
Reusing a model trained on one large task for a new related task, instead of training from scratch.

Q2. Is transfer learning used in your project?
Yes, as feature extraction. Five pre-trained models are used without fine-tuning.

Q3. Which type of transfer learning?
Feature extraction (frozen models). No fine-tuning.

Q4. Name the models.
SSD MobileNet v1 (detection), Face Landmark 68 (landmarks), ResNet face recognition (128-D embedding), MiniFASNetV2 and MiniFASNetV1SE (anti-spoofing).

Q5. Which model does the actual recognition?
The ResNet face recognition model. It outputs a 128-number embedding, compared with Euclidean distance.

Q6. Do you train the model when a user registers?
No. Registration only stores the user's embeddings (encrypted). The model never changes.

Q7. How do you decide who the person is?
Smallest distance must be ≤ 0.5, at least 0.06 better than the second-best person, and the same on 3 consecutive frames.

Q8. How do you stop photo attacks?
Two MiniFASNet models (passive liveness) plus a head-turn challenge verified with the 68 landmarks.

Q9. Why not train your own model?
That needs millions of labelled faces, GPUs and weeks. Transfer learning gives proven accuracy with 5 photos per user on a normal CPU.

Q10. What libraries run the models?
TensorFlow.js (WASM backend) through @vladmandic/face-api for models 1–3; onnxruntime-node for the MiniFASNet models.

Q11. What is an embedding?
A list of numbers that describes a face so that the same person gives close numbers and different people give far numbers.

Q12. What is metric learning?
The training method the ResNet authors used: pull the same person's embeddings together, push different people apart.

14. Remember it in 60 seconds

1 line: My project uses **transfer learning (feature extraction)**: **5 pre-trained, frozen models** find the face, mark 68 points, check it's real, and turn it into **128 numbers**, which are **compared** with registered users. **Nothing is trained.**

Nickname	Real name	One job	File that uses it
Finder	SSD MobileNet v1	Where is the face?	faceEngine.js
Pointer	Face Landmark 68	Where are eyes, nose, jaw? (pose + challenge)	faceEngine.js, faceGeometry.js
Fingerprinter	Face Recognition ResNet	Who is it? (128 numbers)	faceEngine.js → matcher.js
Guards	MiniFASNetV2 + V1SE	Real person or fake?	liveness.js

3 words: Freeze → Extract → Compare

4 models order: Find → Point → Guard → Fingerprint

Don't forget: Registration **stores** numbers; it does **not train**. The models give the **knowledge**; our code gives the **rules**.